COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S3
DATE PERFORMED:	7/16/2026	**DATE SUBMITTED:**	7/19/2026
NAME:	Dimaguiba, Lorenzo James M.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

HANDS-ON ACTIVITY NO. 2.1 Arrays, Pointers, and Dynamic Memory Allocation

1. PROCEDURE OUTPUT

ILO A: Implement static and dynamic memory allocation & ILO B: Create dynamically allocated objects using pointers and arrays:

Screenshot

pointpoint > Dimaguiba_Pointpoint.cpp > ...

```cpp
//Dimaguiba, Lorenzo James M.
#include <iostream>
#include <string.h>

class Student{
    private:
        std::string studentName;
        int studentAge;

    public:
        //constructor
        Student(std::string newName = "LJ Dimaguiba", int newAge = 19){
            studentName = std::move(newName);
            studentAge = newAge;
            std::cout<<"Constructor Called."<<std::endl;
        };

        //deconstructor
        ~Student(){
            std::cout<<"Destructor Called."<<std::endl;
        }

        //Copy Constructor
        Student(const Student &copyStudent){
            std::cout<<"Copy Constructor Called."<<std::endl;
            studentName = copyStudent.studentName;
            studentAge = copyStudent.studentAge;
        }

        //Display Attributes
        void printDetails(){
            std::cout<<this->studentName<<" "<<this->studentAge<<std::endl;
        }
    };

int main(){
    Student student1("Sam Winchester",22); //Something something supernatural
    Student student2(student1);
    Student student3;
    student3 = student2;

    return 0;
}
```

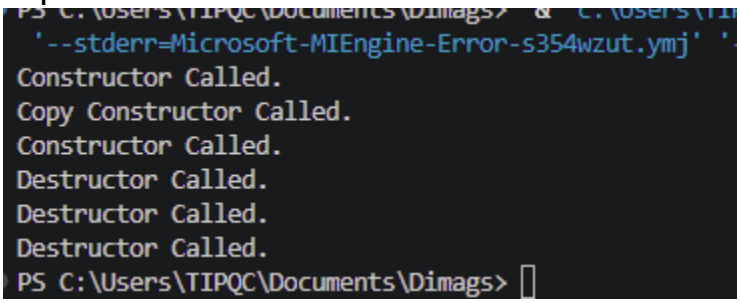
	Output: 
Observation	Once the program begins, as soon as "int main()" runs, it is already on the stack or already the class is called and assigned to the first object "student1", waiting to be initialized. The Constructor function begins to run, declaring new variables according to the input, and printing a confirmation that it ran. Then a new object "student2" was created with "student1" as a parameter. This calls upon the Copy Constructor function, creating a new object with the same properties as "student1". It also prints a confirmation. The final object "student3" is created and without input parameters, it constructs with the default given values. Afterwards, it copies properties from student 2. Since the object is already created, no need to call on the Copy Constructor. As the program comes to an end, the Destructor is called. This applies to every object created and destroyed.

Table 2-1. Initial Driver Program

Screenshot

pointpoint > Dimaguiba_Pointpoint.cpp > Student

```cpp
//Dimaguiba, Lorenzo James M.
#include <iostream>
#include <string.h>

class Student{
    private:
        std::string studentName;
        int studentAge;

    public:
        //constructor
        Student(std::string newName = "LJ Dimaguiba", int newAge = 19){
            studentName = std::move(newName);
            studentAge = newAge;
            std::cout<<"Constructor Called."<<std::endl;
        };

        //deconstructor
        ~Student(){
            std::cout<<"Destructor Called."<<std::endl;
        }

        //Copy Constructor
        Student(const Student &copyStudent){
            std::cout<<"Copy Constructor Called."<<std::endl;
            studentName = copyStudent.studentName;
            studentAge = copyStudent.studentAge;
        }

        //Display Attributes
        void printDetails(){
            std::cout<<this->studentName<<" "<<this->studentAge<<std::endl;
        }
    };

int main(){
    const size_t j = 5;

    Student studentList[j] = {};
    std::string namesList[j] = {"Freddy", "Chica", "Bonnie","Foxy", "Michael"};
    int ageList[j] = {15,16,18,19,16};
    return 0;
}

```

Output:

```
'--stderr=Microsoft-M
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
```

Observation	**Just like before, it starts off normally and this time, there is a constant variable "j" set to 5. Followed by creating a class. The copy constructor is called for each index and loops back to that line until all indices with the default values given. This ends up having 5 objects with the same default values. With the 2 other arrays not really being used by any of the constructors or even the class.**

Table 2-2. Modified Driver Program with Student Lists

Screenshot	

```
pointpoint > Dimaguiba_Pointpoint.cpp > Student > printDetails()
1   //Dimaguiba, Lorenzo James M.
2   #include <iostream>
3   #include <string.h>
4
5   class Student{
6       private:
7           std::string studentName;
8           int studentAge;
9
10      public:
11          //constructor
12          Student(std::string newName = "LJ Dimaguiba", int newAge = 19){
13              studentName = std::move(newName);
14              studentAge = newAge;
15              std::cout<<"Constructor Called."<<std::endl;
16          };
17
18          //deconstructor
19          ~Student(){
20              std::cout<<"Destructor Called."<<std::endl;
21          }
22
23          //Copy Constructor
24          Student(const Student &copyStudent){
25              std::cout<<"Copy Constructor Called."<<std::endl;
26              studentName = copyStudent.studentName;
27              studentAge = copyStudent.studentAge;
28          }
29
30          //Display Attributes
31          void printDetails(){
32              std::cout<<this->studentName<<" "<<this->studentAge<<std::endl;
33          }
34      };
35
36  int main(){
37      const size_t j = 5;
38
39      Student studentList[j] = {};
40      std::string namesList[j] = {"Freddy", "Chica", "Bonnie","Foxy", "Michael"};
41      int ageList[j] = {15,16,18,19,16};
42
43      for (int i = 0; i < j; i++ ){ //Loop A
44          Student *ptr = new Student(namesList[i],ageList[i]);
45          studentList[i] = *ptr;
46      }
47
48      for (int i = 0; i < j; i++ ){ //loop B
49          studentList[i].printDetails();
50      }
51
52      return 0;
53  }
```

	Output:
	`Constructor Called.`
	`Constructor Called.`
	`Constructor Called.`
	`Constructor Called.`
	`Constructor Called.`
	`Constructor Called.`
	`Constructor Called.`
	`Constructor Called.`
	`Constructor Called.`
	`Constructor Called.`
	`Freddy 15`
	`Chica 16`
	`Bonnie 18`
	`Foxy 19`
	`Michael 16`
	`Destructor Called.`
	`Destructor Called.`
	`Destructor Called.`
	`Destructor Called.`
	`Destructor Called.`

Loop A Observation	**Makes use of dynamic allocation. In this case it allocates space for different objects under the same class "Student". It creates new objects for every name and age in the arrays before it; mainly the namesList and ageList arrays. It then sets the values of the main list(studentList) to the newly created objects' values.**
Loop B Observation	**It runs the "printDetails()" function for the original array with the newly assigned values.**
Output Observation	**Once again, with a constant value of 5 in the indices of the first array, it creates new objects with the default values given in the constructor. After which, namesList and ageList are initialized. The first loop allocates memory with a pointer and creates brand new objects under the same class (using "new") and creates objects somewhere in memory to values under namesList and studentList. It now sets the values of the main list(studentList) to the newly created objects' values.The last loop prints all the details with the class method to do so. This in turn uses the constructor function 10 times and the destructor**

	function 5 times.

Table 2-3. Final Driver Program

Screenshot	```
	for (int i = 0; i < j; i++){ //Loop A
	    Student *ptr = new Student(namesList[i],ageList[i]);
	    studentList[i] = *ptr;
	    delete ptr;
	    ptr = nullptr;
	}
	``
	Output:
	``
	Constructor Called.
	Destructor Called.
	Constructor Called.
	Destructor Called.
	Constructor Called.
	Destructor Called.
	Constructor Called.
	Destructor Called.
	Constructor Called.
	Destructor Called.
	Freddy 15
	Chica 16
	Bonnie 18
	Foxy 19
	Michael 16
	Destructor Called.
	Destructor Called.
	Destructor Called.
	Destructor Called.
	Destructor Called.
	``

Observation	Works exactly the same as before. The only problem is that the pointer under loop A allocates memory to create a new object. However, after running, the program ends but the memory is still there taking up space. It had to be deallocated. Hence, the delete function to free up that used space. It is proven with the extra deconstructor calls. And the pointer is set to NULL to point it to nothing afterwards to avoid any future problems.

Table 2-4. Modifications/Corrections Necessary

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1:

```cpp
#include <iostream>
#include <string>
#include <iomanip>

float totalSum(float, float);

class Fruit{
    private:
        std::string fName;
        float fPrice;
        int fQuantity;

    public:
    //constructor
        Fruit(std::string name = "placeholder", float price = 0, int quantity = 0){
            fName = name;
            fPrice = price;
            fQuantity = quantity;
            std::cout<<"Constructor Called"<<std::endl;
        }
    //destructor
        ~Fruit(){
            std::cout<<"Destructor Called"<<std::endl;
        }

    //copy constructor
        Fruit(const Fruit &copyFruit){
            fName = copyFruit.fName;
            fPrice = copyFruit.fPrice;
            fQuantity = copyFruit.fQuantity;
            std::cout<<"Copy Constructor Called"<<std::endl;
        }

    //copy assignment operator
        Fruit& operator = (const Fruit &copyFruit){
            if (this != &copyFruit){
                fName = copyFruit.fName;
                fPrice = copyFruit.fPrice;
                fQuantity = copyFruit.fQuantity;

            }

            return *this;
    }

    //display that thang

        void printDetails(){
            std::cout<<std::fixed<<std::setprecision(2)<<this->fName<<" Price: "<<this->fPrice<<" Quantity: "<<this->fQuantity<<"x"<<std::endl;
        }
    //Sum
        float totalSum(){
            return fPrice * fQuantity;
        }

    };
```

```cpp
class Vegetable{
    private:
        std::string vName;
        float vPrice;
        int vQuantity;

    public:
    //constructor
        Vegetable(std::string name = "placeholder", float price = 0, int quantity = 0){
            vName = name;
            vPrice = price;
            vQuantity = quantity;
            std::cout<<"Constructor Called"<<std::endl;
        }
    //destructor
        ~Vegetable(){
            std::cout<<"Destructor Called"<<std::endl;
        }

    //copy constructor
        Vegetable(const Vegetable &copyVegetable){
            vName = copyVegetable.vName;
            vPrice = copyVegetable.vPrice;
            vQuantity = copyVegetable.vQuantity;
            std::cout<<"Copy Constructor Called"<<std::endl;
        }

    //copy assignment operator
        Vegetable& operator = (const Vegetable &copyVegetable){
            if (this != &copyVegetable){
                vName = copyVegetable.vName;
                vPrice = copyVegetable.vPrice;
                vQuantity = copyVegetable.vQuantity;

            }

            return *this;
    }

    //display that thang

        void printDetails(){
            std::cout<<std::fixed<<std::setprecision(2)<<this->vName<<" Price: "<<this->vPrice<<" Quantity: "<<this->vQuantity<<"x"<<std::endl;
        }

    //Sum

        float totalSum(){
            return vPrice * vQuantity;
        }

    };
```

Discussion/Analysis:
Contains all the necessary properties and methods, the Big Three, and the sum. Using encapsulation on some parts (private and public), this applies for both Fruits and Vegetables. Their use however may vary, depending on the type of method used later.

Problem 2 -3:

```
int main(){

    float fruitSum = 0;
    float veggieSum = 0;

    //Fruit
    const int PRODUCE = 2;
    Fruit* fruitGroceryList[PRODUCE] = {}; //a pointer  for dynamic allocation
    std::string fruitNames[PRODUCE] = {"Apple", "Banana"};
    float fruitPrices[PRODUCE] = {10,10};
    int fruitQuantity[PRODUCE] = {7,8};

    //Vegetables
    Vegetable* veggieGroceryList[PRODUCE] = {}; //a pointer so we can delete lettuce later
    std::string veggieNames[PRODUCE] = {"Broccoli", "Lettuce"};
    float veggiePrices[PRODUCE] = {60,50};
    int veggieQuantity[PRODUCE] = {12,10};

    for (int i = 0; i < PRODUCE; i++){
        Fruit* ptr = new Fruit(fruitNames[i], fruitPrices[i], fruitQuantity[i]);
        fruitGroceryList[i]= ptr;
        fruitSum += ptr->totalSum();
    }

    for (int i = 0; i < PRODUCE; i++){
        Vegetable* ptrV = new Vegetable(veggieNames[i], veggiePrices[i], veggieQuantity[i]);
        veggieGroceryList[i]= ptrV;
        veggieSum += ptrV->totalSum();
    }

    //Display
    for (int k = 0; k < PRODUCE; k++){
        fruitGroceryList[k]->printDetails();
        veggieGroceryList[k]->printDetails();
    }

    std::cout<<"Total Sum: "<<std::fixed<<std::setprecision(2)<<totalSum(veggieSum,fruitSum)<<"php"<<std::endl;
```

Discussion/Analysis:
Since there are 2 classes, and arrays only accept only one type of element, a grocery list for each of the types of produce is created. How it works, instead of creating an array of an element, it makes an array of multiple POINTERS. This is then put into a loop, creating new objects and pointing to those objects. The pointer used to create the new objects before copying to the array is destroyed. This repeats till the array is filled the sum is added to a total for both fruits and veggies.

TotalSum Function:

```
float totalSum(float, float);
```

```
float totalSum(float aPrice, float bPrice){
    return aPrice + bPrice;
}
```

Discussion/Analysis:
Finds the total sum from both fruit and veggie sums.

Problem 4:

```
//Lettuce Deallocate Lettuce (pun intended)
delete veggieGroceryList[1];
veggieGroceryList[1] = nullptr;


//Deallocate everything else since we used dynamic allocation for the classes
for (int i = 0; i < PRODUCE; i++){
    delete fruitGroceryList[i];
    delete veggieGroceryList[i];
}

return 0;
```

Discussion/Analysis:
Took the index of the lettuce, used "delete", then pointed it to "nullptr" so it points to nothing. Before the program ends, it is also good practice to deallocate the rest. Same has been done for the rest of the arrays, focused on deleting the Fruit and Vegetable objects (which is why it was looped to access them).

Output:

```
Constructor Called
Constructor Called
Constructor Called
Constructor Called
Apple Price: 10.00 Quantity: 7x
Broccoli Price: 60.00 Quantity: 12x
Banana Price: 10.00 Quantity: 8x
Lettuce Price: 50.00 Quantity: 10x
Total Sum: 1370.00php
Destructor Called
Destructor Called
Destructor Called
Destructor Called
```

Analysis:
Shows when the objects are created, with the constructor and when they were destroyed. Displayed all elements properly. Including the total sum of all.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **(Conceptual) Arrays and pointers.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Researched at what arrays can store, discovered that it can store pointers too. All code is done by hand and only knowledge was applied.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.